

The Hopfield Model

Associative Memory & The Model

Chapter 2.1 & 2.2

Hertz, Krogh, Palmer (1991)
Introduction to the Theory of Neural Computation

Today's Roadmap

From a memory problem to its mathematical foundation

1

Why Associative Memory?

The fruit fly problem of neural networks

2

Configuration Space & Attractors

A geometric picture of memory

3

Setting Up the Model

Notation, dynamics, asynchronous updates

4

Storing One Pattern

Hebb rule in its simplest form

5

Storing Many Patterns

Hebb rule and stability analysis

6

Crosstalk & Storage Capacity

How many patterns can we store?

7

The Energy Function

Hopfield's central insight

8

Spurious States

When memory misbehaves

Section 2.1

The Associative Memory Problem

"The fruit fly problem of neural networks"

The Basic Problem

A simple but profound challenge

The Problem

Store a set of **p patterns ξ^μ** in such a way that when presented with a new pattern ζ , the network responds by producing whichever one of the stored patterns most closely resembles ζ .

Two essential properties:

Content-Addressable

Recall the whole memory from a fragment.

Example: "evolution" $\rightarrow$ all info about Darwin

Error-Correcting

Recover the original despite noise.

Example: " $E = mc^3$ " $\rightarrow$ still recalls Einstein

Measuring Similarity: Hamming Distance

How different are two patterns?

Definition (Eq. 2.1)

$$d_H(\xi^\mu, \zeta) = \sum_i [\xi_i^\mu(1-\zeta_i) + (1-\xi_i^\mu)\zeta_i]$$

Counts the number of bits that differ

Per-site contribution:

ξ_i	ζ_i	Result
0	0	0 (same)
0	1	1 (differ)
1	0	1 (differ)
1	1	0 (same)

Example (N = 5)

$$\xi^\mu = (1, 0, 1, 1, 0)$$

$$\zeta = (1, 1, 1, 0, 0)$$

Differences: (-, X, -, X, -)

$$d_H = 2$$

In ± 1 notation:

$$d_H = \frac{1}{2} \sum_i (1 - \xi_i S_i)$$

Equivalent — but easier for math.

Inner product $\xi \cdot S$ = **overlap**.

Configuration Space & Attractors

A geometric picture of memory

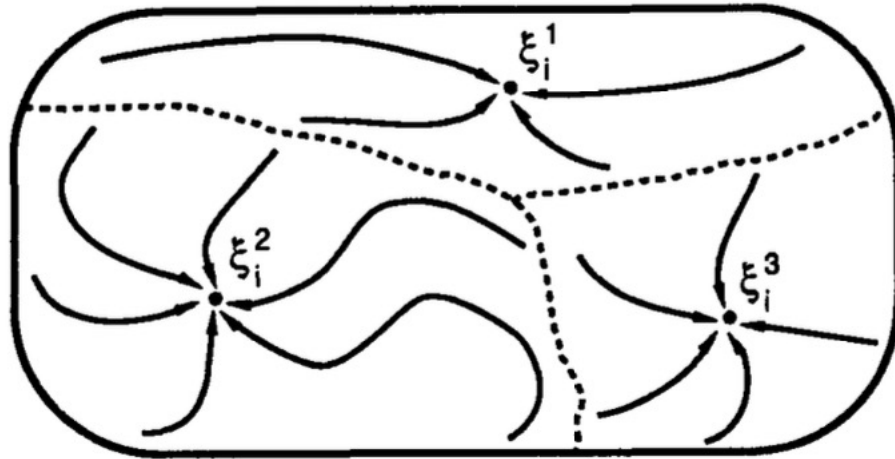


FIGURE 2.2 Schematic configuration space of a model with three attractors.

Configuration space

All possible network states ($\{S_i\}$).

Attractors

Stored patterns ξ^u are stable points.

Basins of attraction

Each attractor pulls in a region of space.

*The network's dynamics **flows** any starting point into one of the attractors.*

Section 2.2

The Model

Building Hopfield's network step by step

Notation: Why Switch to ± 1 ?

A small change with big consequences

$$S_i = 2 n_i - 1$$

State mapping

biological	spin (S_i)	meaning
0	-1	not firing
1	+1	firing

Three reasons:

1

Mathematical symmetry

Multiplication has group structure on $\{-1, +1\}$

2

Hebb rule becomes natural

$w_{ij} \propto \xi_i \xi_j \rightarrow$ same sign +1, opposite -1

3

Connects to Ising model

Statistical mechanics tools become available

Same biology — cleaner math. All physics is the same; only the labels change.

The Update Rule

How a single neuron decides its next state

Eq. (2.2) General update rule

$$S_i \leftarrow \text{sgn} \left(\sum_j w_{ij} S_j - \theta_i \right)$$

net input from other neurons minus threshold $\rightarrow$ sign decides on/off

Eq. (2.3) sgn function

$$\begin{aligned} \text{sgn}(x) &= +1 && \text{if } x \geq 0 \\ &= -1 && \text{if } x < 0 \end{aligned}$$

Eq. (2.4) Simplified ($\theta = 0$)

$$S_i \leftarrow \text{sgn} \left(\sum_j w_{ij} S_j \right)$$

the rule we'll actually use

Why $\theta_i = 0$?

Random patterns have +1 and -1 with equal probability $\rightarrow$ no built-in bias.

A non-zero threshold would tip the network toward one state.

(For sparse / asymmetric patterns, an appropriate ϑ would matter — see §3.3.)

Synchronous vs Asynchronous Updating

When do we update which neuron?

SYNCHRONOUS

All neurons update simultaneously.

Needs:

- central clock / pacemaker
- precise timing

Risks:

- oscillations / limit cycles
- no convergence guarantee

ASYNCHRONOUS ✓

One neuron updates at a time.

Two ways:

- (a) pick i randomly each step
- (b) each neuron fires independently with constant probability

Properties:

- natural for brain & hardware
- **guaranteed convergence**

This book — and most analyses — adopt the asynchronous choice.

Storing One Pattern

The simplest case — building intuition

Goal: make pattern ξ_i a fixed point

Eq. (2.5) Stability condition

$$\text{sgn} \left(\sum_j w_{ij} \xi_j \right) = \xi_i \quad \text{for all } i$$

A choice that works: Eq. (2.7)

$$w_{ij} = (1/N) \xi_i \xi_j$$

Why it works:

$$\begin{aligned} \sum_j w_{ij} \xi_j &= (1/N) \sum_j \xi_i \xi_j \xi_j = (1/N) \xi_i \sum_j 1 \\ &= \xi_i \end{aligned}$$

(used $\xi_j^2 = 1$)

Error Correction

If fewer than half the bits are wrong,
the right ones win the **majority vote** in $\sum_j w_{ij} S_j \rightarrow$ bit i is corrected.

⚠ Reversed State

$-\xi$ is also stable!

Configuration space splits into two equal basins (Fig. 2.4).

Many Patterns: The Hebb Rule

Just add the contributions

Eq. (2.9) Hebb Rule (generalized)

$$W_{ij} = (1/N) * \sum_{\mu} (\xi_i^{\mu} * \xi_j^{\mu})$$

Sum over all patterns $\mu = 1, \dots, p$ — superposition principle

"Neurons that fire together, wire together."

— Donald Hebb (1949)

Differences from Hebb's original idea:

ξ_i^{μ}	ξ_j^{μ}	$\xi_i^{\mu} \xi_j^{\mu}$ in Eq. (2.9)	Hebb's original rule
+1	+1	+1 (strengthen)	✓ strengthen
+1	-1	-1 (weaken)	no change
-1	+1	-1 (weaken)	no change
-1	-1	+1 (strengthen!)	X not biological

Stability Analysis: The Crosstalk Term

Why doesn't the system always work?

Net input to neuron i when system is in pattern v :

Eq. (2.12) After splitting the sum on μ :

$$h_i^v = \underbrace{\xi_i^v}_{\text{signal}} + \underbrace{(1/N) \sum_j \sum_{\{\mu \neq v\}} (\xi_i^\mu * \xi_j^\mu * \xi_j^v)}_{\text{crosstalk (noise from other patterns)}}$$

SIGNAL ξ_i^v

Comes from the $\mu = v$ term.
Exactly the bit value we want.
Drives the network toward ξ^v .

CROSSTALK (noise)

Interference from all other $p - 1$ patterns.
If its magnitude < 1 : harmless.
If > 1 : it flips the sign of h_i^v .

The Key Question

How big does p have to be before crosstalk overwhelms signal?

Distribution of the Crosstalk

Random patterns + Central Limit Theorem

Define $C_i^v = -\xi_i^v \times (\text{crosstalk term})$ Eq. (2.13)

- $C_i^v < 0$ → crosstalk helps (same sign as signal)
- $0 < C_i^v < 1$ → opposite but harmless
- $C_i^v > 1$ → bit flips → pattern is unstable

Statistics (random patterns)

C_i^v = sum of $\sim Np$ independent ± 1 terms

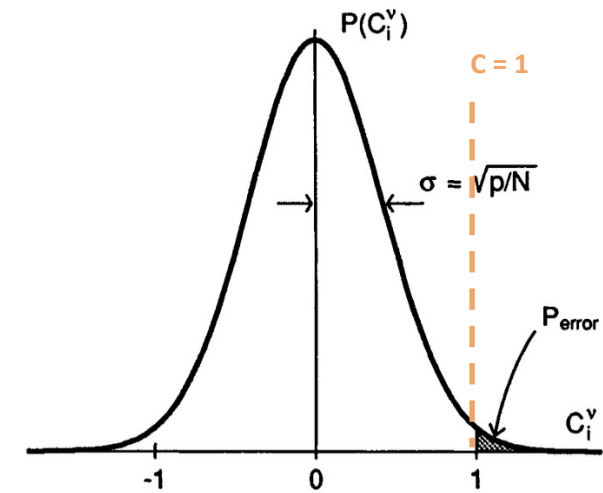
By CLT → Gaussian:

- mean = 0
- variance $\sigma^2 = p / N$

Eq. (2.15)

$$P_{\text{error}} = \frac{1}{2} [1 - \text{erf}(\sqrt{N/2p})]$$

Distribution of C_i^v (Fig. 2.5)



Gaussian, $\sigma^2 = p/N$

Storage Capacity

How many patterns can we cram in?

Table 2.1 Capacity vs. tolerated bit error rate

P_error (per bit)	p_max / N	Comment
0.001	0.105	very strict
0.0036	0.138*	famous capacity limit
0.01	0.185	1% bit error
0.05	0.37	loose
0.1	0.61	very loose

Three different capacity definitions:

1. Few errors per bit

$p_{\max} \approx 0.138 N$ (avalanche limit)

2. Most patterns recalled perfectly

$p_{\max} \approx N / (2 \log N)$

3. All patterns recalled perfectly

$p_{\max} \approx N / (4 \log N)$

The 0.138 N limit is the AVALANCHE threshold

Below 0.138 N: small initial errors stay small (errors don't cascade).

Above 0.138 N: one bit flip triggers another → whole memory collapses.

Derived rigorously in §2.5 using statistical mechanics (replica method).

Special Case: Orthogonal Patterns

When 'optimal' becomes useless

If patterns happen to be orthogonal:

$$\sum_j (\xi_j^u - \xi_j^v) = 0 \quad (\mu \neq v) \quad \text{Eq. (2.22)}$$

→ Crosstalk vanishes. $C_i^v = 0$ for all i, v . Sounds perfect... but:

The Trap

If we store N orthogonal patterns:

$$w_{ij} = \delta_{ij}$$

Each unit only connects to itself.

- **every state is stable**
- *no error correction at all!*

The Lesson

Capacity is not just "# of stable patterns".

It's the # of patterns with a **finite basin of attraction**.

Otherwise the network is a static lookup, not an associative memory.

Hopfield's Central Insight

The Energy Function

Network dynamics = rolling downhill

Stored memories = valleys at the bottom

The Energy Function

Borrowing a powerful idea from physics

Eq. (2.24)

$$H = -1/2 * \sum_{i,j} (w_{ij} S_i S_j)$$

The central claim:

As the network evolves, H never increases.

- ✓ Attractors = local minima of H
- ✓ Network always converges
- ✓ Symmetric weights are required: $w_{ij} = w_{ji}$

The Energy Landscape

A useful — if idealized — picture

Imagine a hilly surface above configuration space.

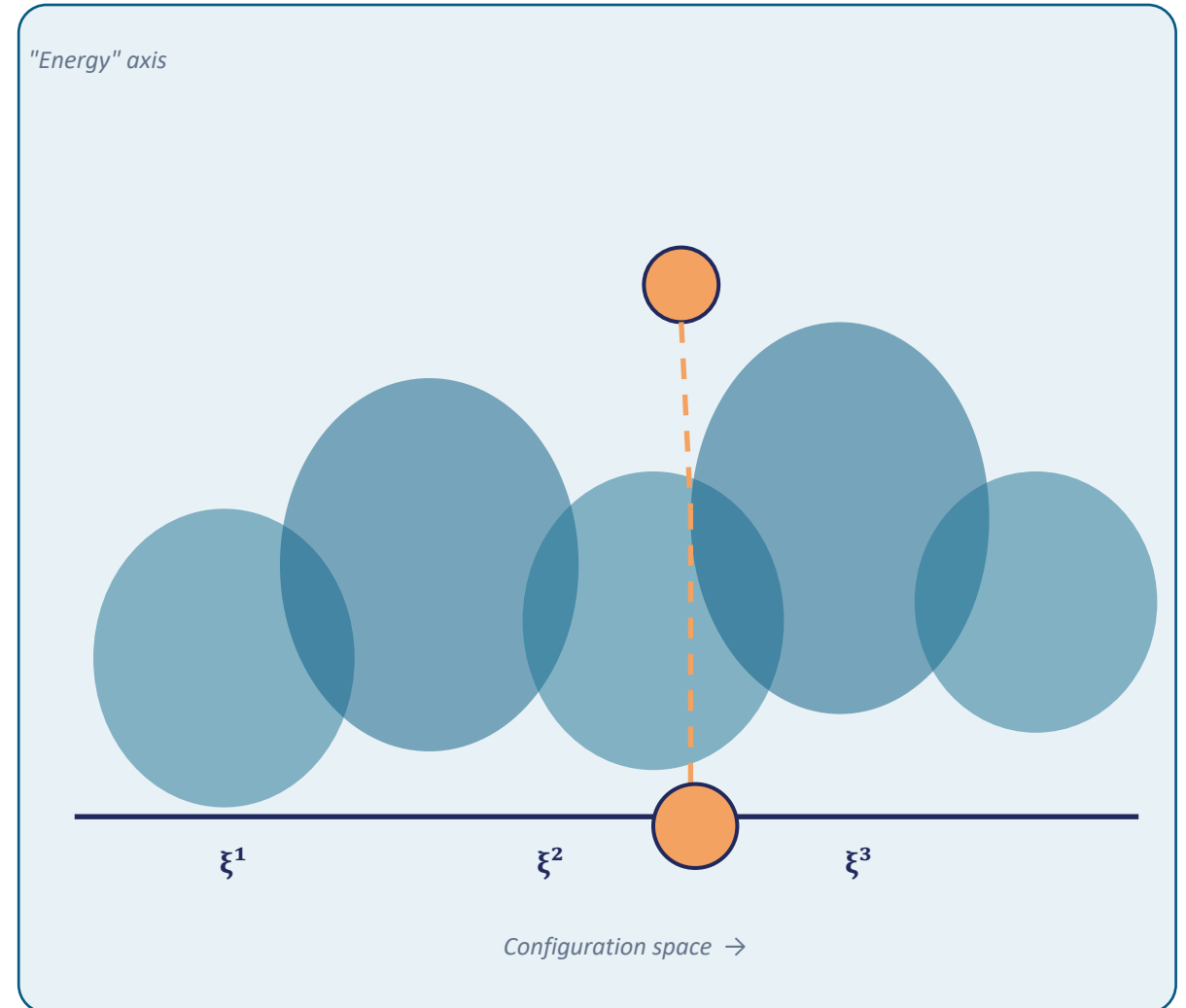
- Each network state $\{S_i\} \rightarrow$ one point on the surface
- Height of the surface = energy H of that state
- Update rule = step that decreases height

Memories sit at local minima.

Basins of attraction are the valleys around them.

Caveat:

Real configuration space is discrete — the 2^N vertices of an N -cube. The smooth surface is an analogy, not literal.



Why H Decreases (Sketch of Proof)

Eq. (2.27) — the heart of the chapter

Suppose neuron i flips: $S_i \rightarrow S_i' = -S_i$. Compute the energy change.

Eq. (2.27)

$$H' - H = 2 S_i \sum_j w_{ij} S_j - 2 w_{ii}$$

(symmetric weights $w_{ij} = w_{ji}$ are essential here)

First term is negative

The flip happens because
 $S_i' = \text{sgn}(\sum_j w_{ij} S_j) = -S_i$

So S_i and $\sum_j w_{ij} S_j$
have **opposite signs**.

Their product < 0 .

Second term is negative

Hebb rule gives
 $w_{ii} = (1/N) \sum_{\mu} (\xi_i^{\mu})^2 = p/N > 0$

So $-2 w_{ii} < 0$.

Both terms negative $\rightarrow H$ decreases.

Reverse Direction: Energy $\rightarrow$ Hebb Rule

A general method for the rest of the book

Idea: define H so that minima sit at the patterns we want to store.

Eq. (2.30) Sum of squared overlaps

$$H = - (1/2N) \sum_{\mu} \left(\sum_i \xi_i^{\mu} S_i \right)^2$$

Maximum overlap with each pattern $\rightarrow$ minimum energy



expand the square

Eq. (2.31) After expansion

$$H = - \frac{1}{2} \sum_{ij} \left[\left(\frac{1}{N} \sum_{\mu} \xi_i^{\mu} \xi_j^{\mu} \right) \right] S_i S_j = - \frac{1}{2} \sum_{ij} w_{ij} S_i S_j$$

Hebb rule (2.9) drops out automatically.

General Recipe

Write your problem as an energy function quadratic in S . Read off w_{ij} from the coefficient of $S_i S_j$. $\rightarrow$ Used heavily in Chapter 4 (TSP, etc.).

Spurious States

Memories we didn't ask for

The Hebb rule produces the desired attractors ξ^u ("retrieval states") — but also others:

1. Reversed

$$-\xi^u$$

Same energy as ξ^u .

From the symmetry

$$S_i \leftrightarrow -S_i.$$

Fix: use one sign bit as reference and flip the rest.

2. Mixture

$$\text{sgn}(\pm \xi^1 \pm \xi^2 \pm \xi^3)$$

Combinations of **odd** numbers (3, 5, 7...) of stored patterns.

Even numbers can sum to 0.

Hamming distance $N/4$ from each component.

3. Spin glass

uncorrelated

Random local minima with no overlap with any ξ^u .

Appear when p is large.

Connected to spin glass physics — see §2.5.

✓ Spurious states have small basins of attraction → hard to fall into in practice. Tricks (finite T , biased patterns) reduce them further.

Putting It All Together

The logical flow of Chapter 2.1 & 2.2

1. Memory Problem

Store p patterns; retrieve from partial input

2. Spin notation

$S_i \in \{-1, +1\}$, asynchronous sgn dynamics

3. Hebb Rule

$$w_{ij} = (1/N) \sum \xi_i^u \xi_j^u$$

4. Stability

signal + crosstalk decomposition

5. Capacity $\approx 0.138 N$

via Gaussian crosstalk + erf

6. Energy Function H

monotonically decreasing $\rightarrow$ memories at minima

Key insight: symmetric weights + asynchronous sgn dynamics $\Rightarrow H$ is a Lyapunov function $\Rightarrow$ memory works.

Key Takeaways

Five things to remember

1

Memory as dynamics

An associative memory is a dynamical system whose attractors are the stored patterns.

2

Hebb rule = superposition

Weights are simply the sum of one-pattern terms. "Fire together, wire together."

3

Signal vs crosstalk

The net input splits cleanly into wanted signal + noise from other patterns. Capacity is just signal-to-noise.

4

0.138 N — a hard limit

Beyond this, an avalanche of bit flips destroys all memories at once. Below, errors stay small.

5

Energy function = the magic

Symmetric $w_{ij} \Rightarrow H$ decreases monotonically $\Rightarrow$ network always converges to a memory. Hopfield's biggest contribution.

Thank You

Hertz, Krogh, Palmer — Introduction to the Theory of Neural Computation (1991)
Chapter 2.1 — The Associative Memory Problem
Chapter 2.2 — The Model